



UPPSALA
UNIVERSITET

Lab 4

Modelling and Verification using UPPAAL

Duc Anh Nguyen
10 Dec 2025

This slide was revised from the previous
year's version.



UPPSALA
UNIVERSITET

Lab goals

- A Hands-on experience with formal modeling and verification of real-time systems.
- Understand Timed Automata model
- Learn how to use UPPAAL.



UPPSALA
UNIVERSITET

Lab Overview

Lab preparation

- Lab session:
 - 17 Dec: 13:15 - 17:00, 10K1125, 10K1205, 10K1206, 10K1210.
 - 19 Dec: 8:15 - 12:00, 4104
- The instructions are on the lab [web page](#)

Hand-in

- A report that answers to the questions for all parts.
- Original model and query files (we will open and test).
- **Deadline:** Friday, 9 Jan, 23:59, 2026.



UPPSALA
UNIVERSITET

UPPAAL

Model-Checking tool for modeling, validation and verification of real-time systems modelled as networks of timed automata

- Developed at Aalborg University, Denmark and Uppsala University, Sweden
- Started 1995, rather mature by now
- GUI in Java, verification engine C++
- Extensive online help. Use it!

Free for private/academic use (but closed-source)

UPPAAL homepage: <http://www.uppaal.org>

UPPAAL documentation: <https://docs.uppaal.org/>



UPPSALA
UNIVERSITET

UPPAAL Installation

Academic Licenses

The Uppaal tool can be used by academic institutions that deliver academic degrees. As [academic use](#), we consider only **You can use 5.0 or 4.0** s delivering academic degrees. In addition, the work or the worker may not be contracted by any non-academic institution. Any other use requires a [commercial license](#).

UPPAAL 5.0

UPPAAL 4.1

UPPAAL 4.0

Other

Windows

macOS

Linux

Arch

Ubuntu

Debian

Fedora

UPPAAL 5.0.0

OS (arch)	Requires Java	Includes Java
macOS (x86_64):	app	dmg
macOS (Aarch64):	app	

Development snapshot 5.1.0-beta5:

OS (arch)	Requires Java	Includes Java
macOS (x86_64):	app	dmg
macOS (Arch64):	app	

Please **register** at [veriaal.dk](#) to obtain a free academic **license key**.

After downloading, follow the [installation-instructions](#) below.



UPPSALA
UNIVERSITET

UPPAAL Installation

UPPAAL Academic License Registration

After the registration, we *immediately* email you the link to your license information.

Fields marked with * are mandatory.

* **Service:** For commercial purposes, register [here](#).
Teachers may create [a license key for students for specific course](#).

* **Name:**

* **E-mail:**

* **Job title:**

* **Organization:**

Web page:

Street address:

City:

Postal code:

Country:

Details:

Application domain?
Running on a grid/cluster environment?

Use your
student
information
here

Privacy policy:

The above is collected solely for [Veriaal ApS](#) statistics and is not shared with anyone else.

The email is used to *validate the organization affiliation* and send the *license information* only.



UPPSALA
UNIVERSITET

Theory review

automata

timed automata

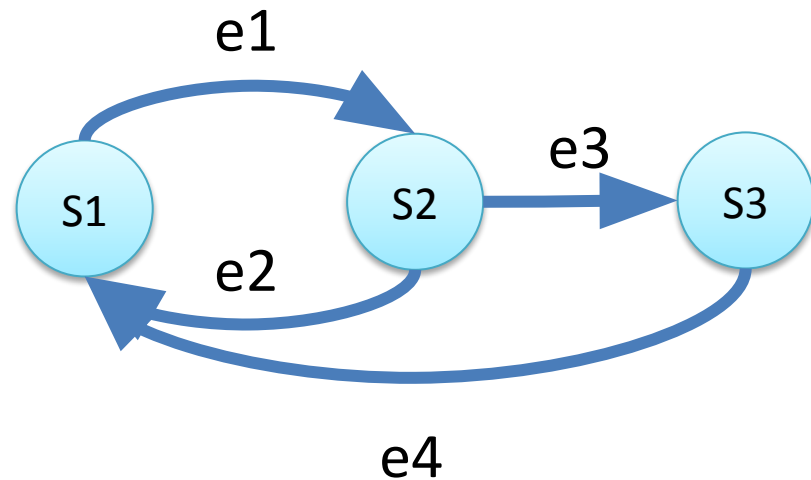


UPPSALA
UNIVERSITET

Finite Automata

States: $S1, S2, S3$

Edges: $e1, e2, e3, e4$



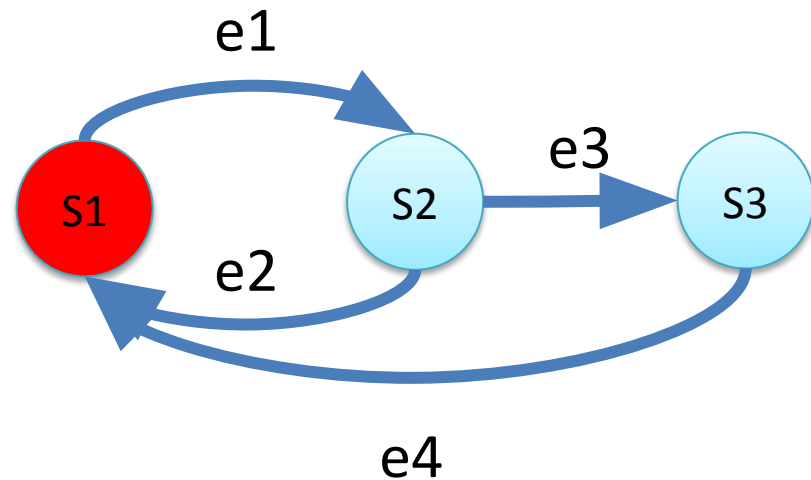


UPPSALA
UNIVERSITET

Finite Automata

States: $S1, S2, S3$

Edges: $e1, e2, e3, e4$



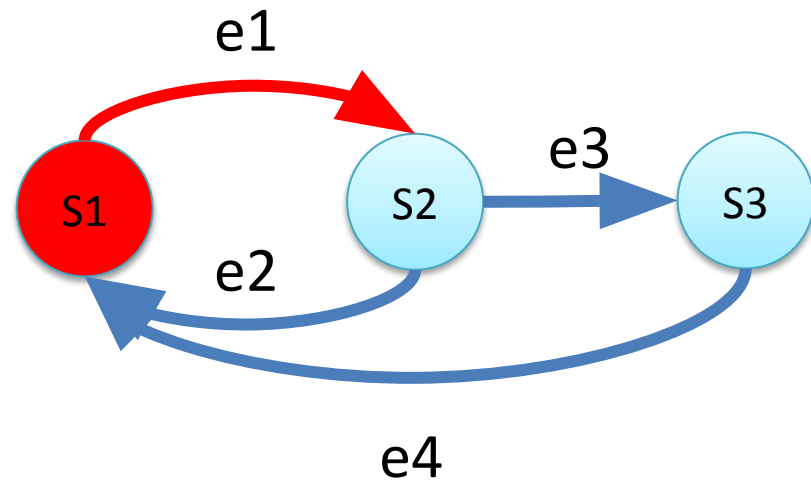


UPPSALA
UNIVERSITET

Finite Automata

States: $S1, S2, S3$

Edges: $e1, e2, e3, e4$



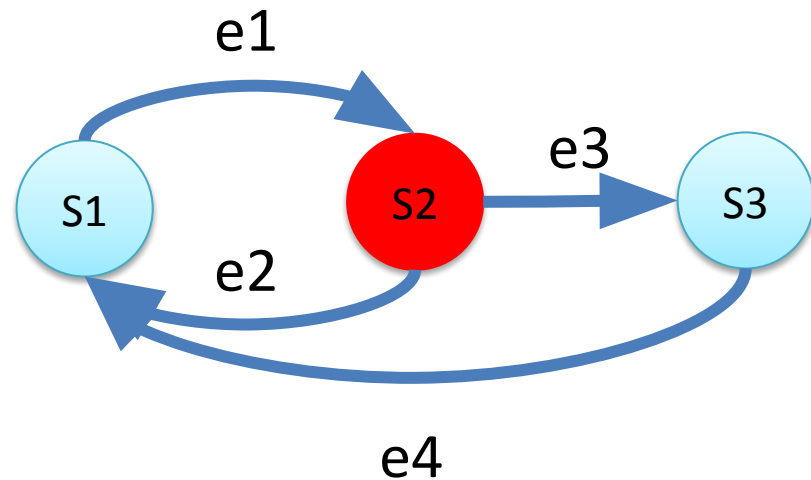


UPPSALA
UNIVERSITET

Finite Automata

States: $S1, S2, S3$

Edges: $e1, e2, e3, e4$



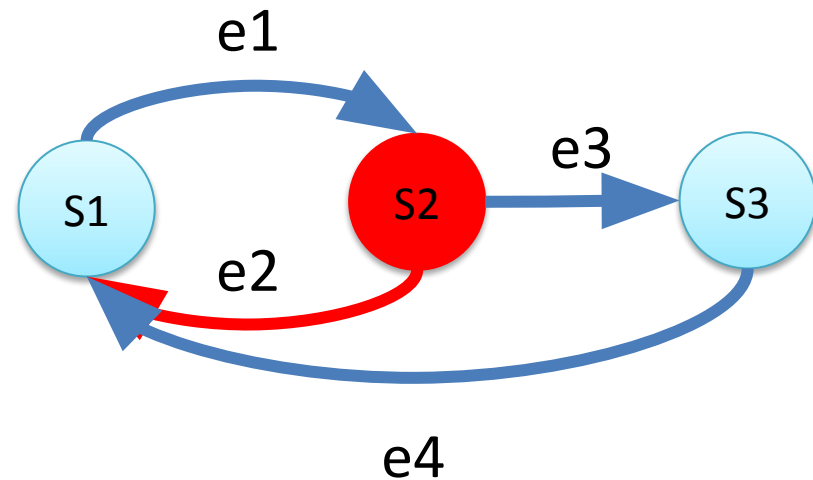


UPPSALA
UNIVERSITET

Finite Automata

States: $S1, S2, S3$

Edges: $e1, e2, e3, e4$



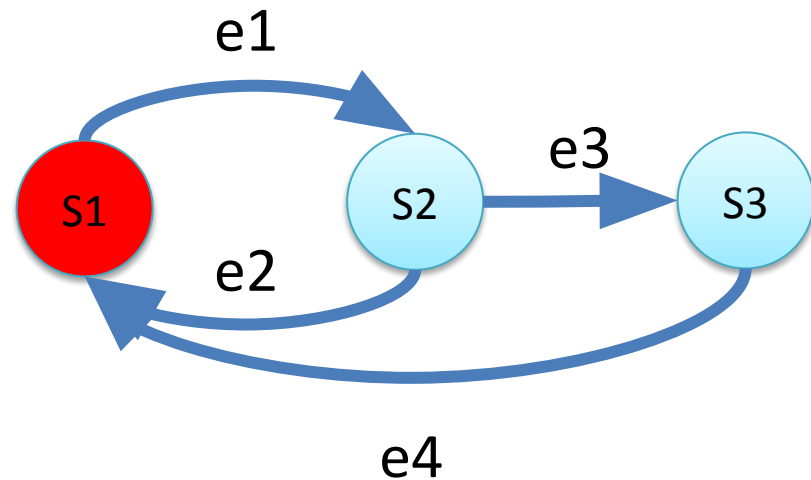


UPPSALA
UNIVERSITET

Finite Automata

States: $S1, S2, S3$

Edges: $e1, e2, e3, e4$



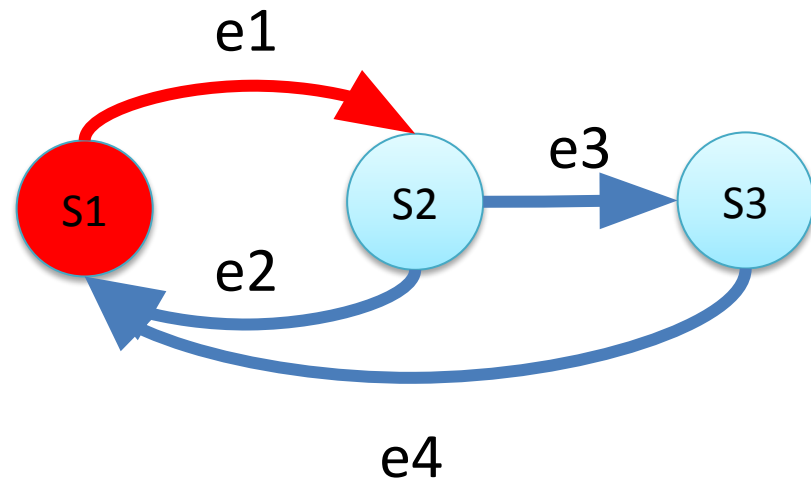


UPPSALA
UNIVERSITET

Finite Automata

States: $S1, S2, S3$

Edges: $e1, e2, e3, e4$



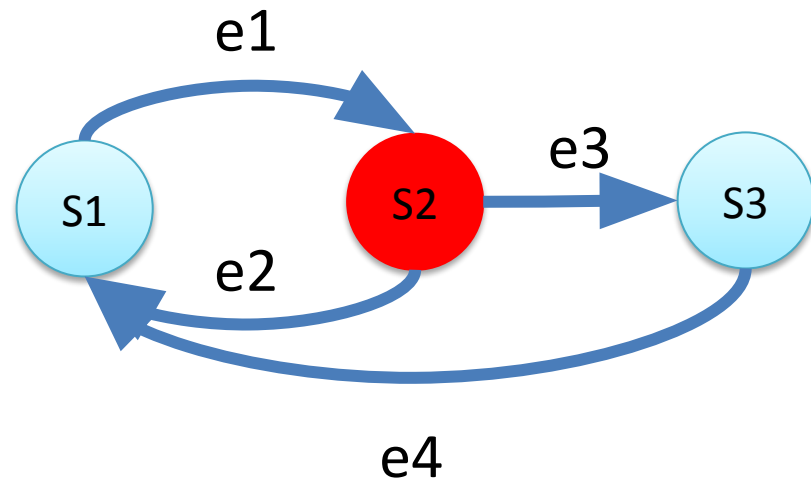


UPPSALA
UNIVERSITET

Finite Automata

States: $S1, S2, S3$

Edges: $e1, e2, e3, e4$



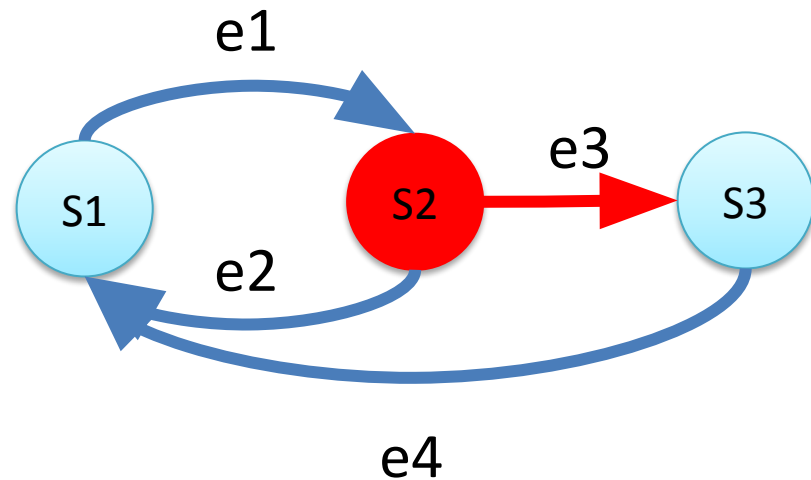


UPPSALA
UNIVERSITET

Finite Automata

States: $S1, S2, S3$

Edges: $e1, e2, e3, e4$



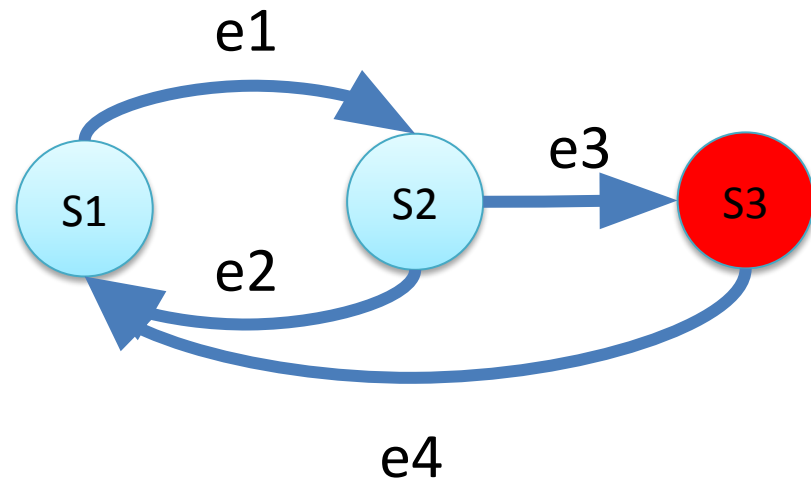


UPPSALA
UNIVERSITET

Finite Automata

States: $S1, S2, S3$

Edges: $e1, e2, e3, e4$



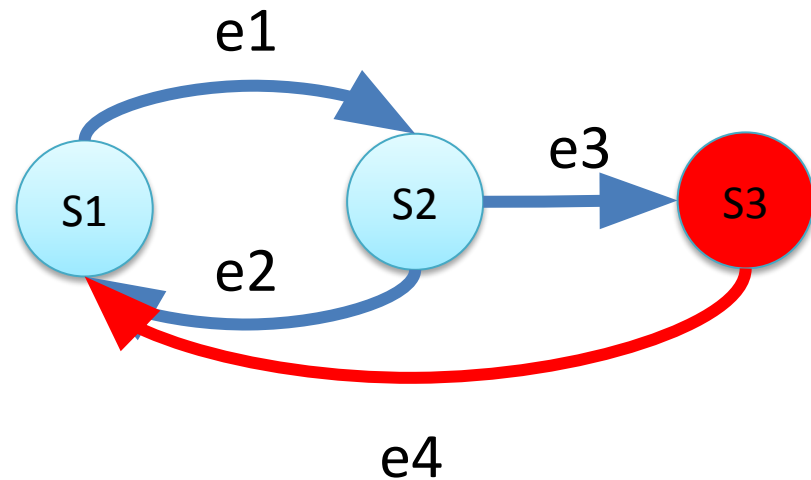


UPPSALA
UNIVERSITET

Finite Automata

States: $S1, S2, S3$

Edges: $e1, e2, e3, e4$



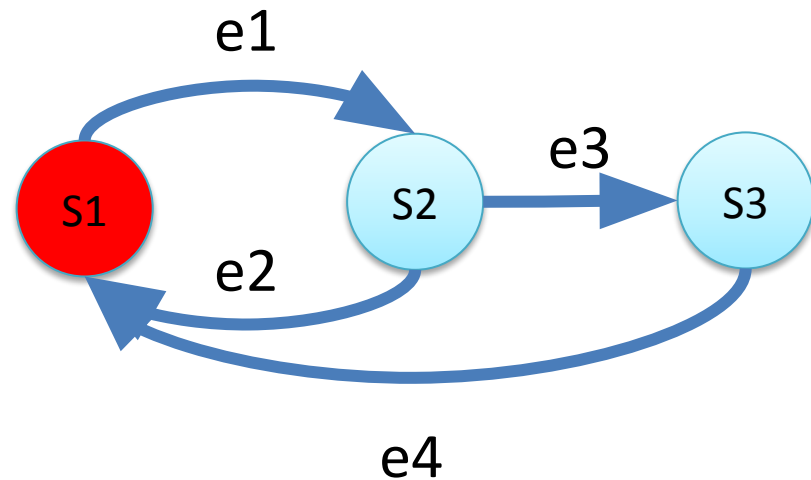


UPPSALA
UNIVERSITET

Finite Automata

States: $S1, S2, S3$

Edges: $e1, e2, e3, e4$



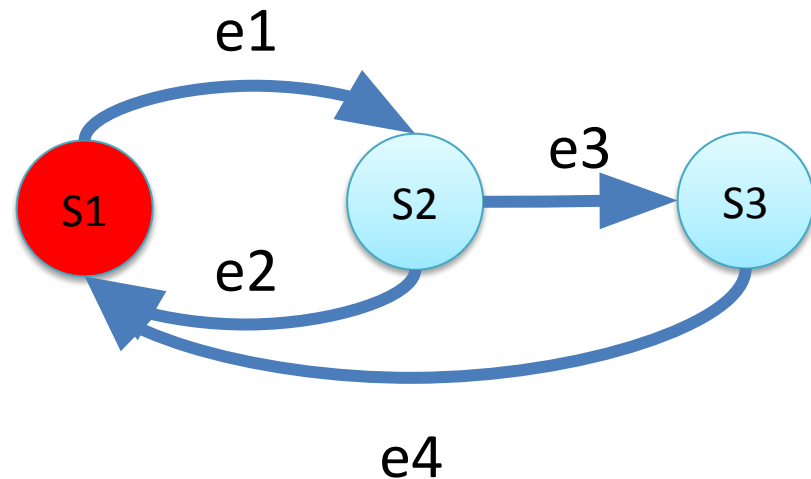


UPPSALA
UNIVERSITET

Finite Automata

States: $S1, S2, S3$

Edges: $e1, e2, e3, e4$



We can abstract programs using finite automata.



UPPSALA
UNIVERSITET

Finite Automata

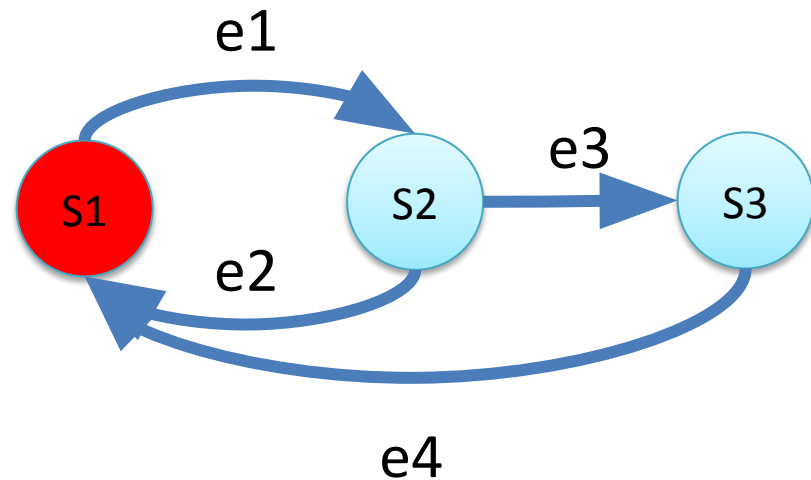
```
while true:
```

```
    if sum < 100:
```

```
        sum := sum + 20;
```

```
    else: Null;
```

```
end while;
```





UPPSALA
UNIVERSITET

Finite Automata

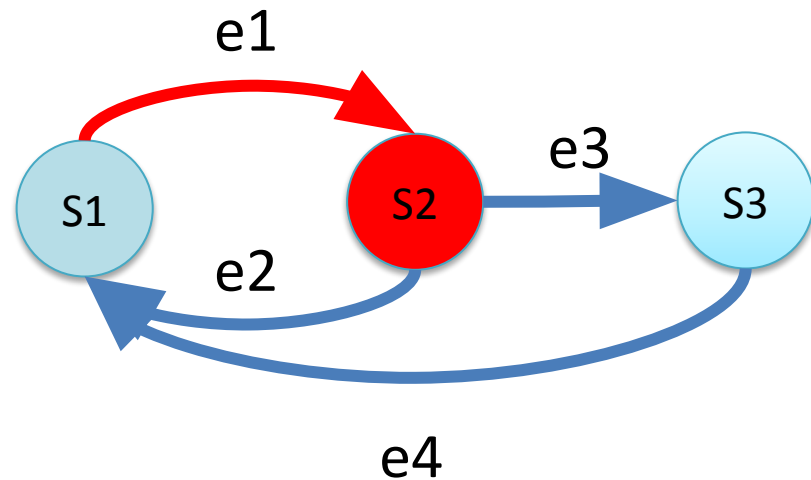
while true:

if sum < 100:

sum := sum + 20;

else: Null;

end while;

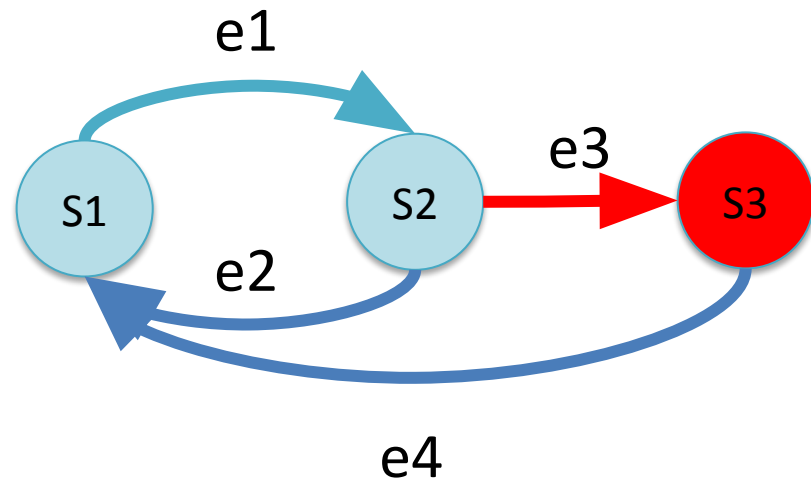




UPPSALA
UNIVERSITET

Finite Automata

```
while true:  
  if sum < 100:  
    sum := sum + 20;  
  else: Null;  
end while;
```





Finite Automata

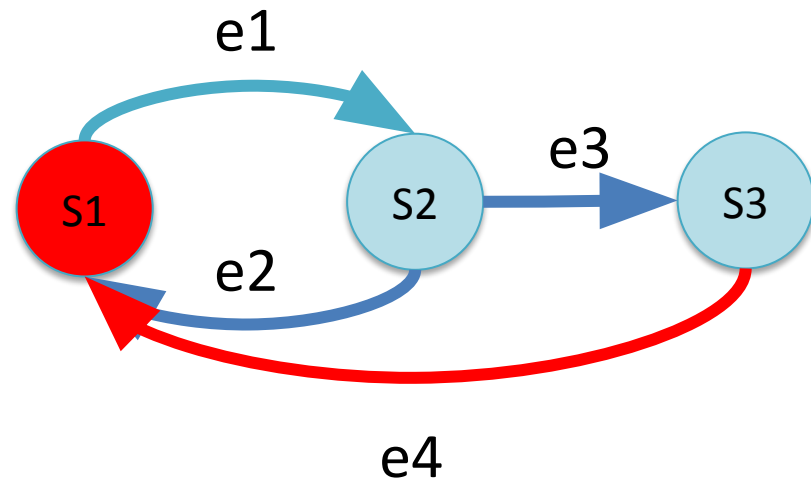
```
while true:
```

```
    if sum < 100:
```

```
        sum := sum + 20;
```

```
    else: Null;
```

```
end while;
```





UPPSALA
UNIVERSITET

Finite Automata

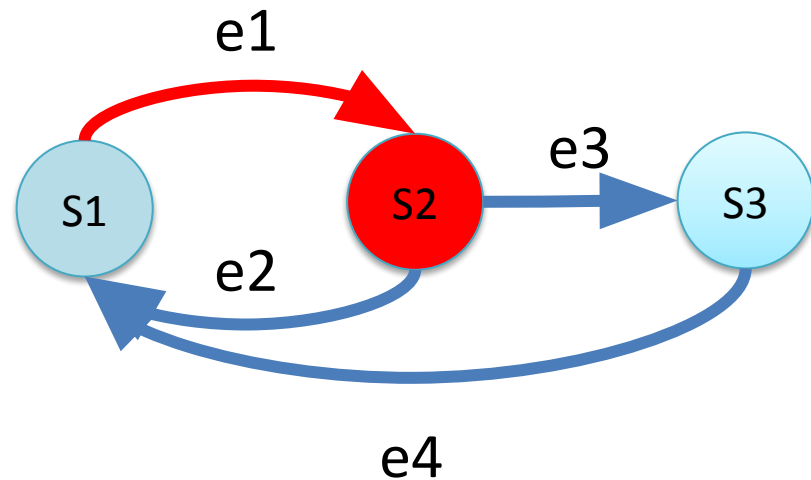
while true:

if sum < 100:

sum := sum + 20;

else: Null;

end while;

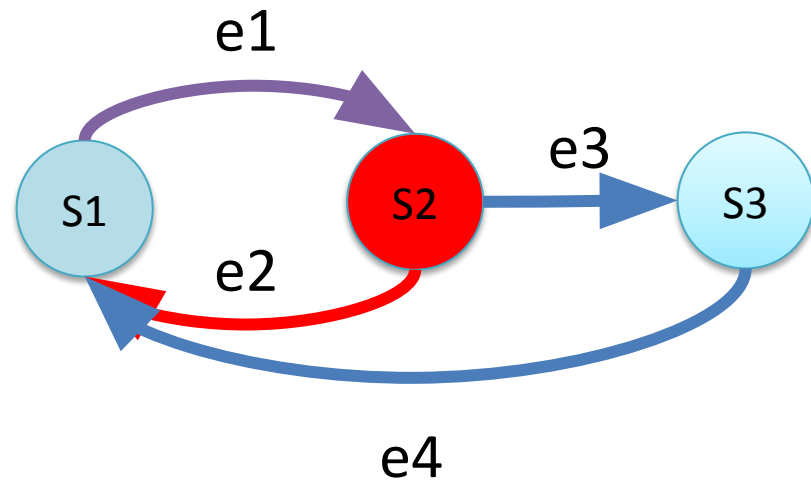




UPPSALA
UNIVERSITET

Finite Automata

```
while true:  
    if sum < 100:  
        sum := sum + 20;  
    else: Null;  
end while;
```





UPPSALA
UNIVERSITET

Finite Automata

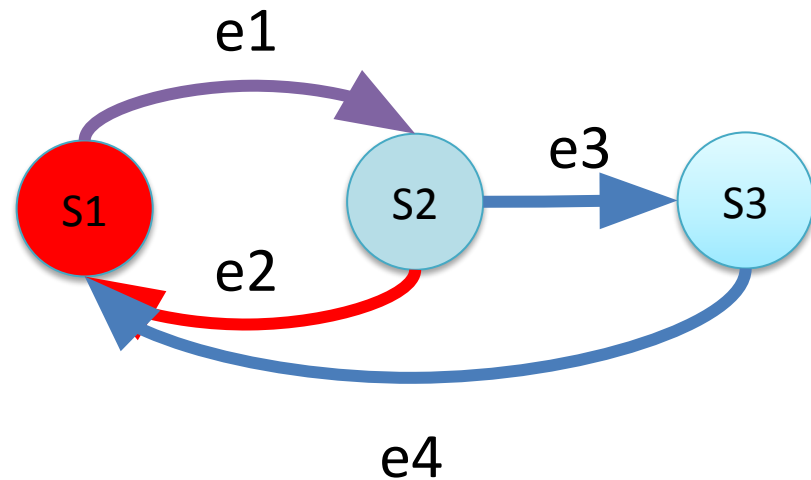
```
while true:
```

```
    if sum < 100:
```

```
        sum := sum + 20;
```

```
    else: Null;
```

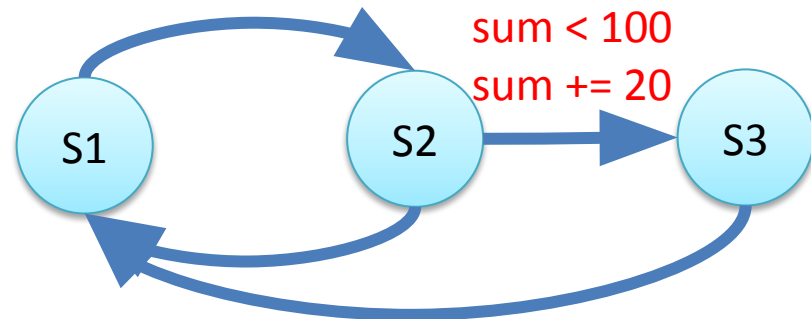
```
end while;
```





Assignments and Guards

```
while true:  
    if sum < 100:  
        sum := sum + 20;  
    else: Null;  
end while;
```

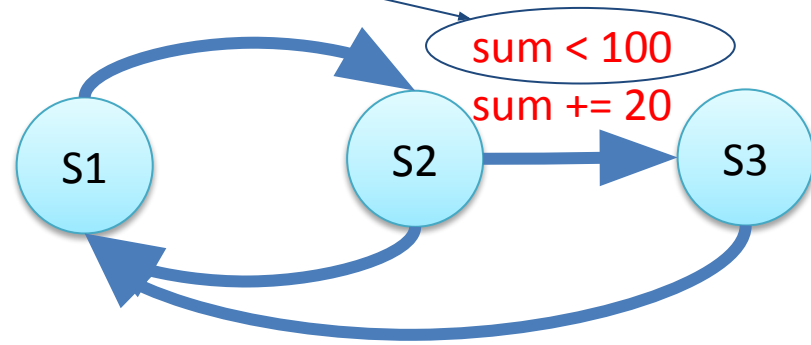




Assignments and Guards

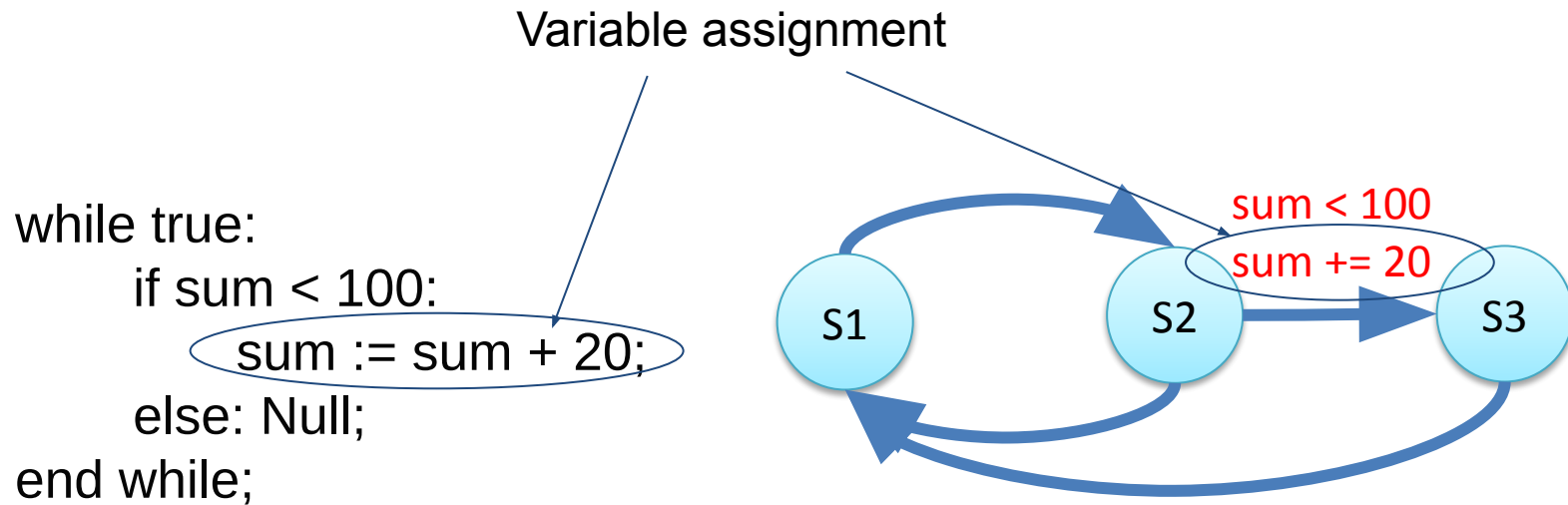
Guard (Condition)

```
while true:  
  if sum < 100:  
    sum := sum + 20;  
  else: Null;  
end while;
```





Assignments and Guards

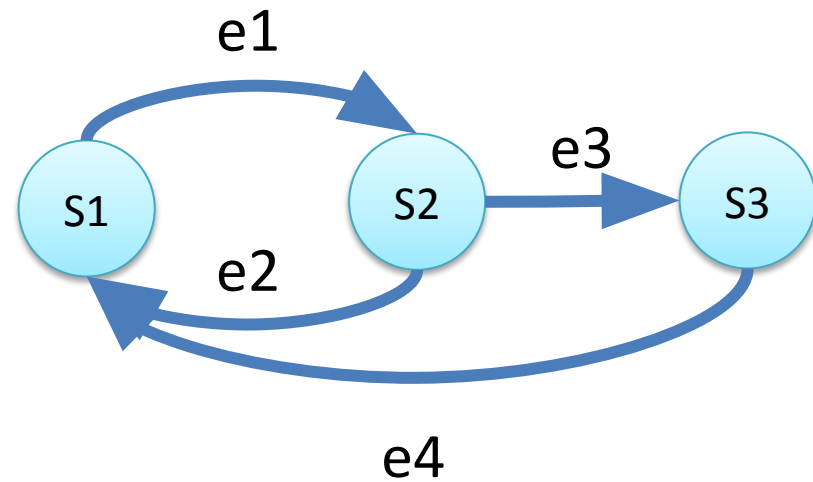




UPPSALA
UNIVERSITET

Finite Automata Systems

P1

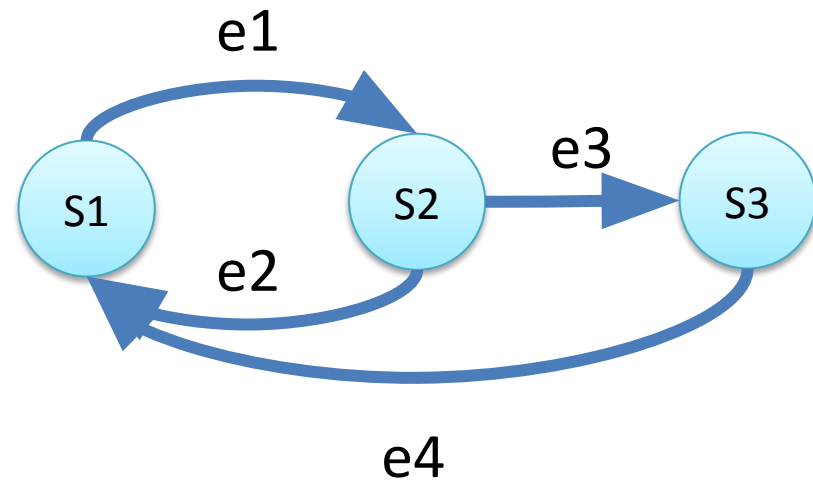




UPPSALA
UNIVERSITET

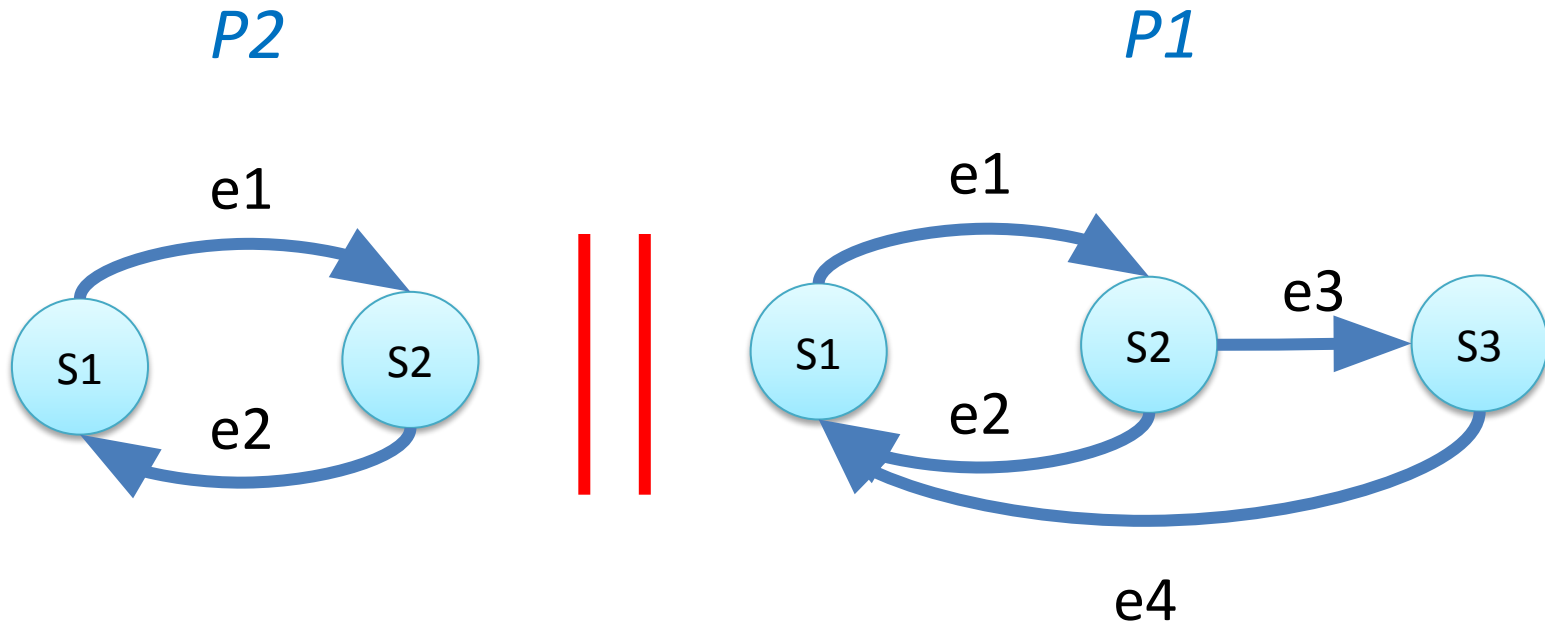
Finite Automata Systems

P1





Finite Automata Systems

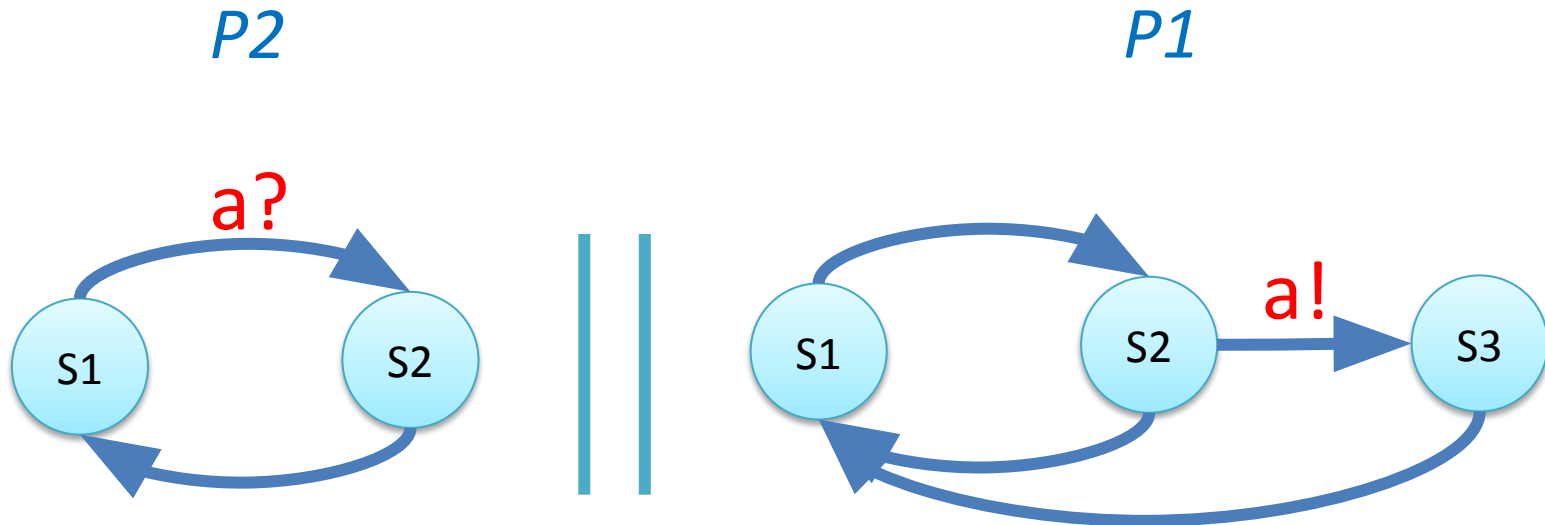


Concurrent systems

P1, *P2* can communicate



Rendezvous Communication



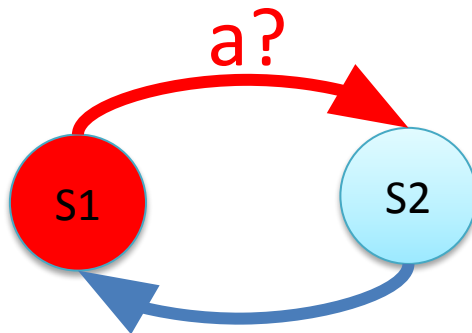
P1 (a!): Process P1 actively sends the signal a when it transitions from S2 to S3

P2 (a?): Process P2 passively waits to receive the signal a to transition from S1 to S2.



Rendezvous Communication

P2



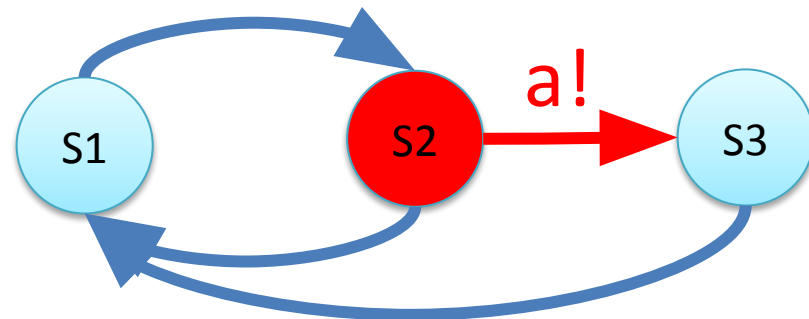
while true:

accept put;

counter++;

end while;

P1



while true:

if sum < 100:

buffer.put;

sum := sum + 20;

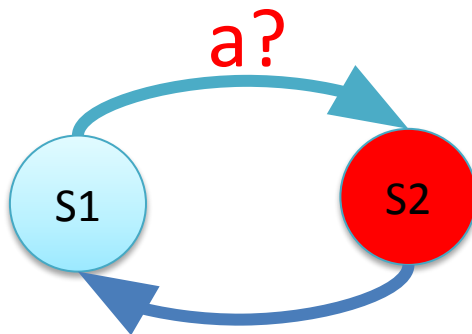
else: Null;

end while;



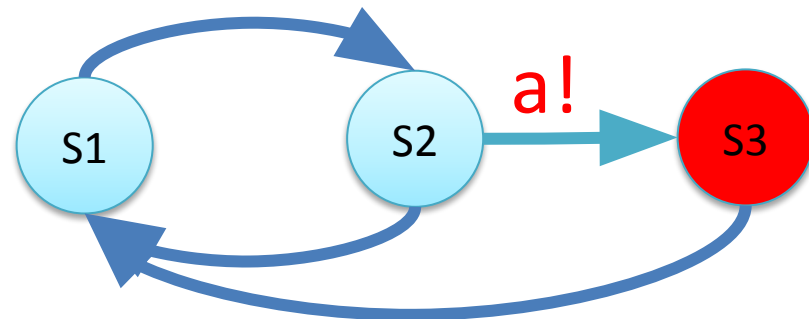
Rendezvous Communication

P2



```
while true:  
  accept put;  
  counter++;  
end while;
```

P1



```
while true:  
  if sum < 100:  
    buffer.put;  
    sum := sum + 20;  
  else: Null;  
end while;
```



UPPSALA
UNIVERSITET

Timed Automata

Timed Automata = Finite Automata + Clock

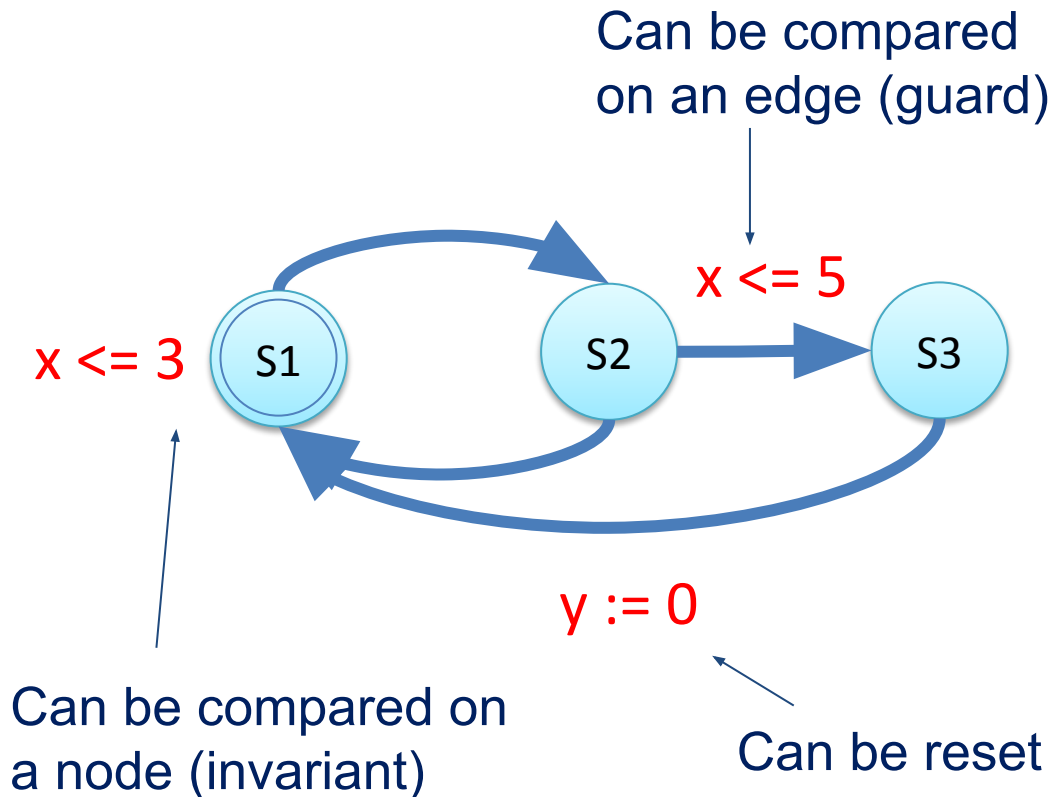


Timed Automata

Clocks: x, y

Clocks have real values.

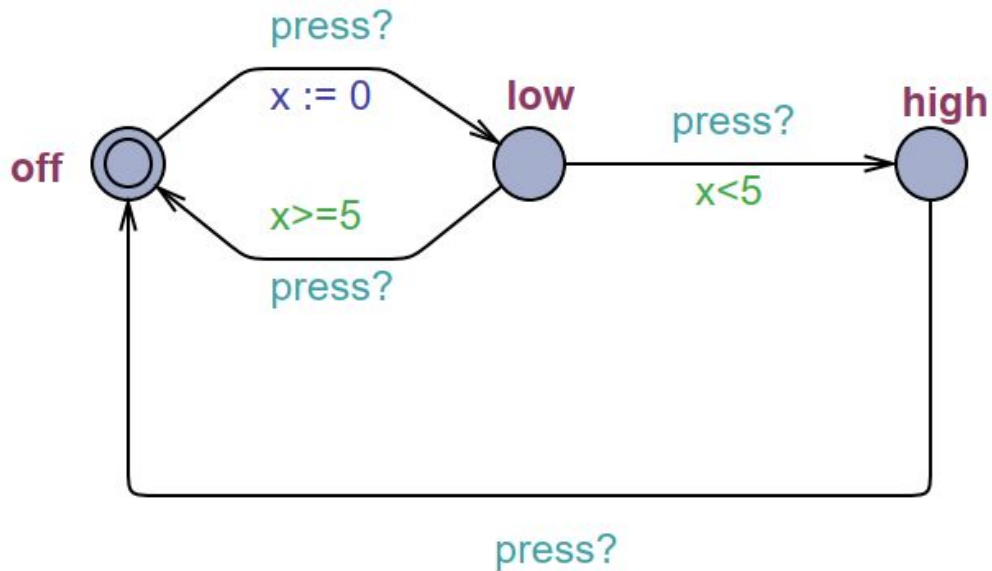
- All clock variables increase at the same pace
- Can be reset and compared





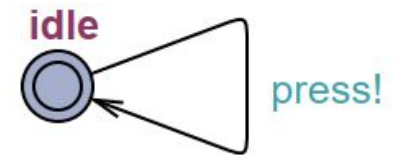
UPPSALA
UNIVERSITET

Timed Automata Systems



Lamp automaton

||

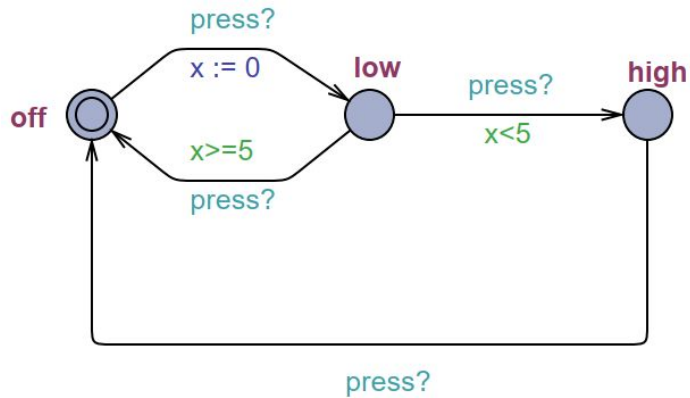


User automaton



UPPSALA
UNIVERSITET

System States



Lamp automaton

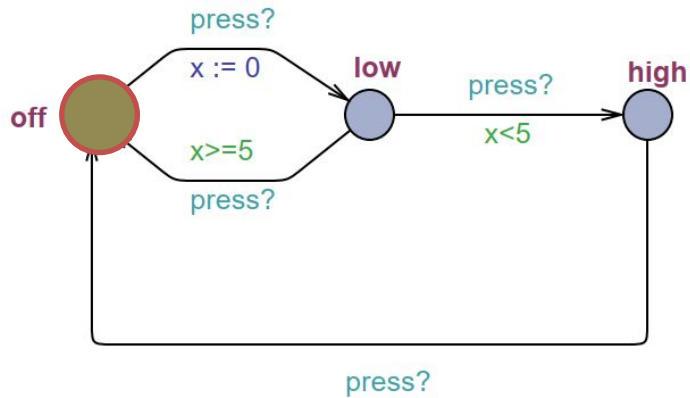


User automaton



UPPSALA
UNIVERSITET

System States

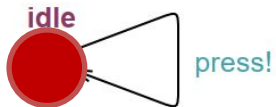


Lamp automaton

State1



$x \geq 0$ (variables)

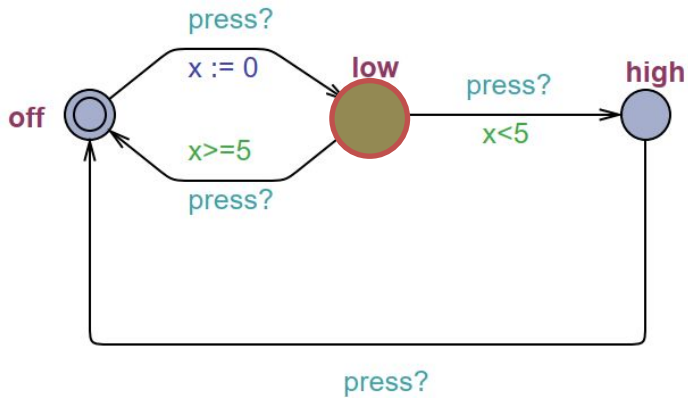


User automaton

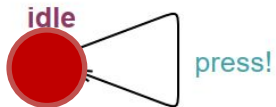


UPPSALA
UNIVERSITET

System States



Lamp automaton



User automaton

State1 Off idle $x \geq 0$ (variables)

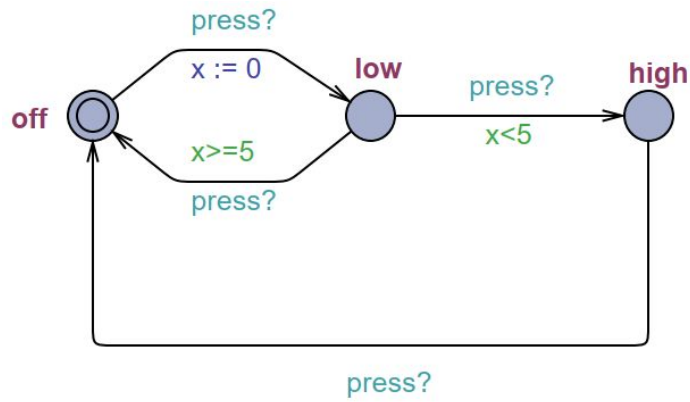
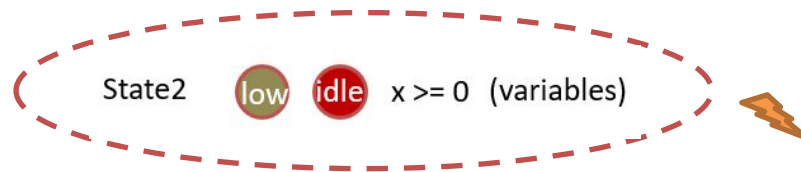


State2 low idle $x \geq 0$ (variables)



UPPSALA
UNIVERSITET

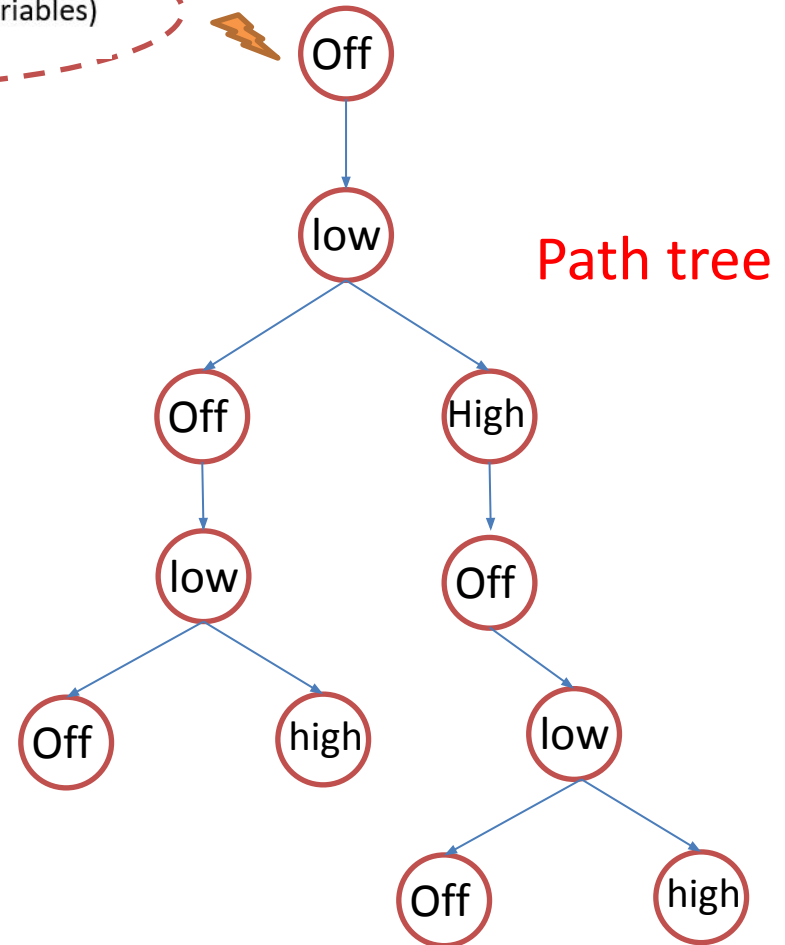
Trace (path)



Lamp automaton



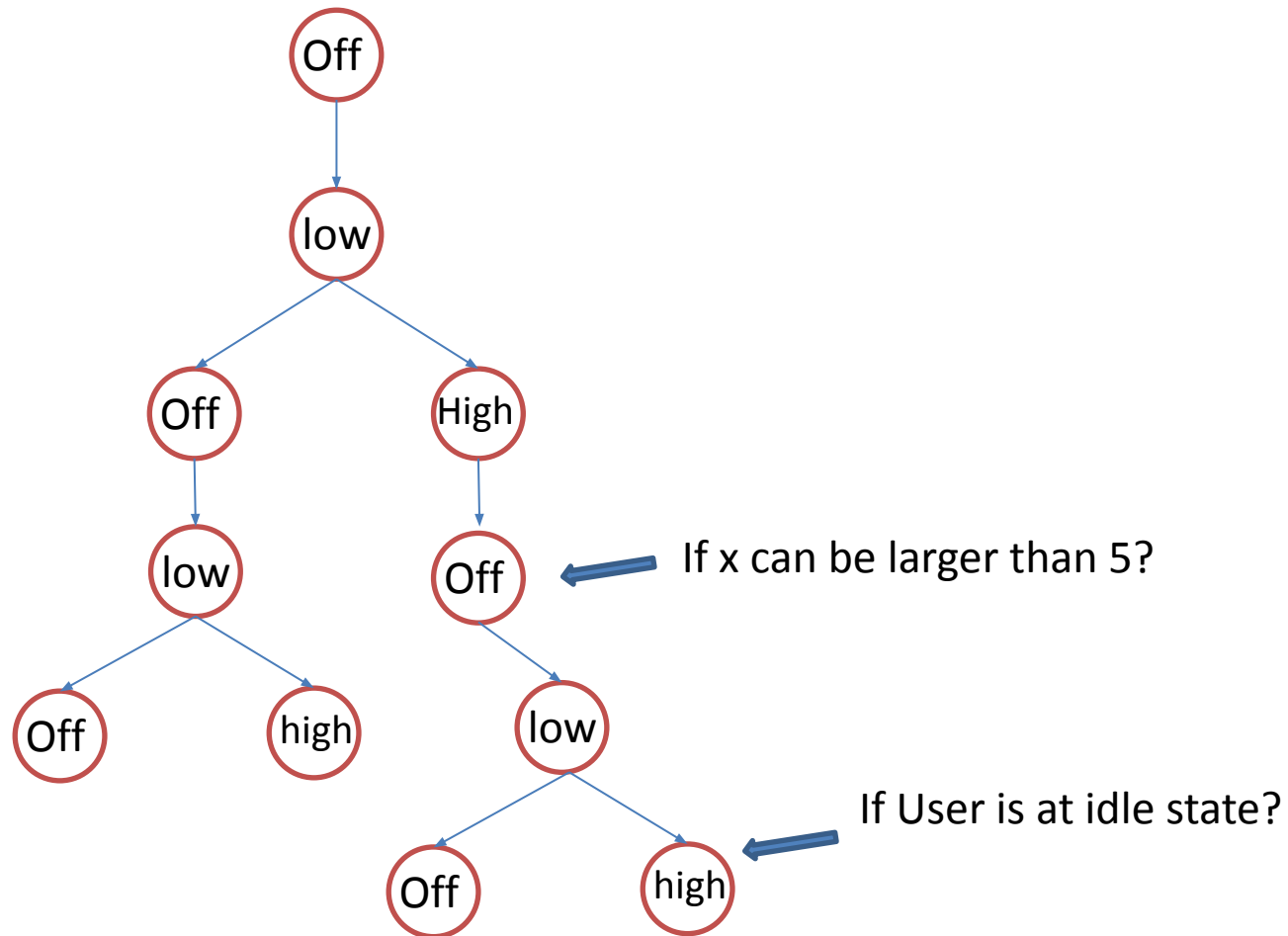
User automaton





Trace (path)

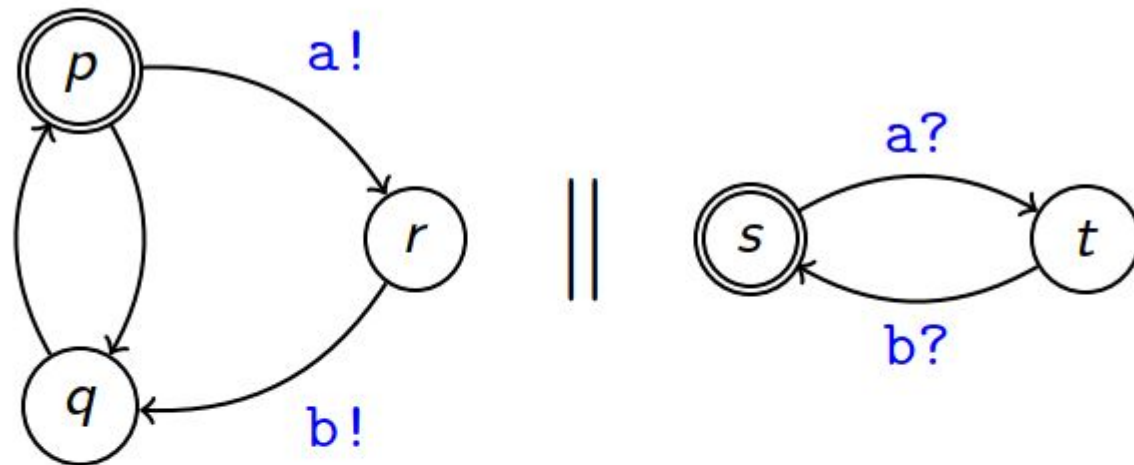
Path tree





Model Checking in Uppaal

Does a model *satisfy* some property φ ?



Automaton A

Automaton B

Property: “Does $A.r$ imply $B.t$?”

- ▶ $\varphi := A[]$ ($A.r$ imply $B.t$)
- ▶ Means: “In each state of each trace, B is in t whenever A in r ”

Is *satisfied* in above example



Model Checking in Uppaal

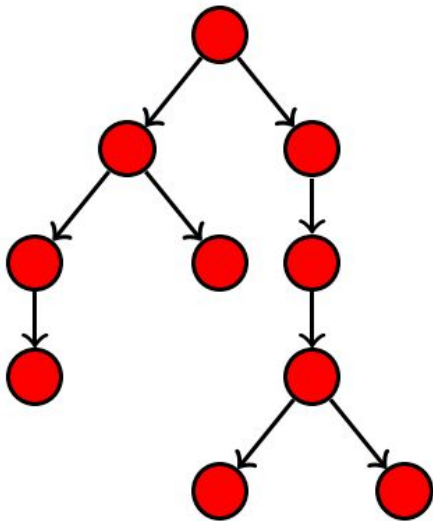
- $E\langle\rangle p$: there exists a path where p eventually holds.
- $A[] p$: for all paths p always holds.
- $E[] p$: there exists a path where p always holds.
- $A\langle\rangle p$: for all paths p will eventually hold.



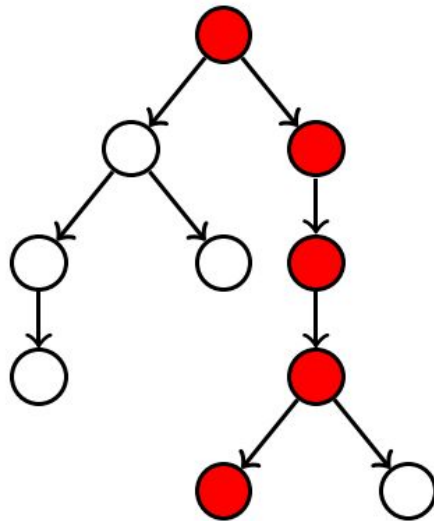
UPPSALA
UNIVERSITET

Model Checking in Uppaal

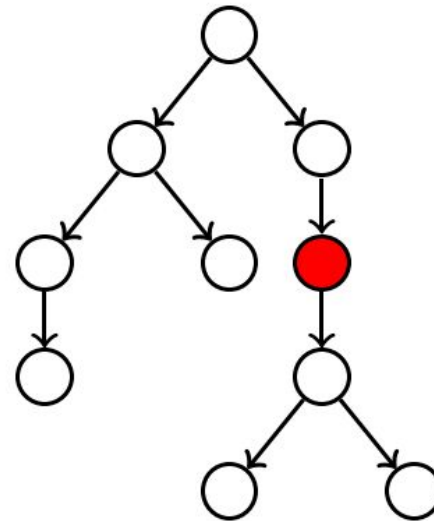
$A[] p$



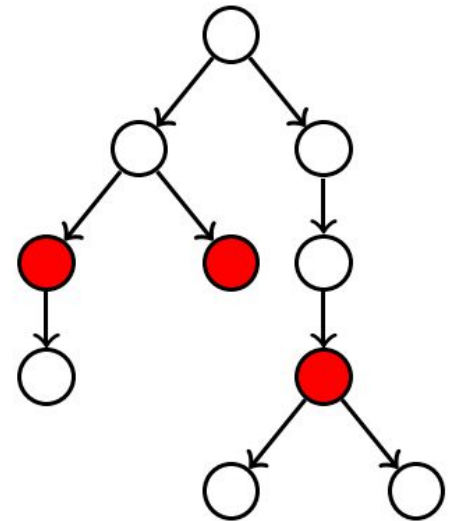
$E[] p$



$E<> p$

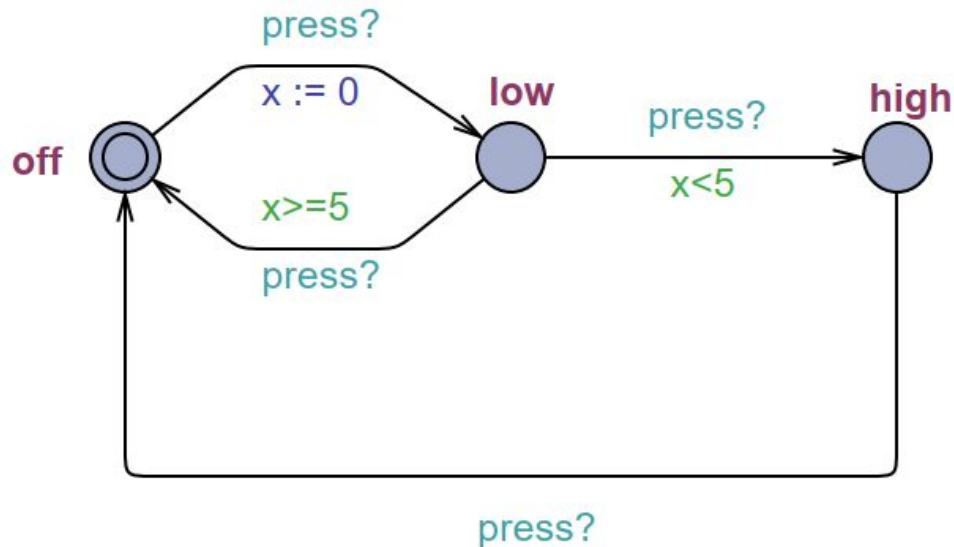


$A<> p$

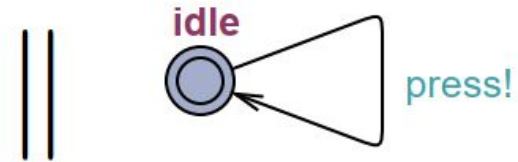




Model Checking in Uppaal



Lamp automaton



User automaton

- $A[]$ not deadlock
- $E \langle \rangle x \geq 10$
- $E[]$ User.idle and Lamp.low



Example

Process 1

```
req1=1;  
turn=2;  
while(turn!=1 && req2!=0);  
// critical section:  
job1();  
req1=0;
```

Process 2

```
req2=1;  
turn=1;  
while(turn!=2 && req1!=0);  
// critical section:  
job2();  
req2=0;
```



Example

Process 1

idle:

req1=1;

want:

turn=2;

wait:

while(turn!=1 && req2!=0);

CS:

//critical section

job1();

req1=0;

//and return to idle

Process 2

idle:

req2=1;

want:

turn=1;

wait:

while(turn!=2 && req1!=0);

CS:

//critical section

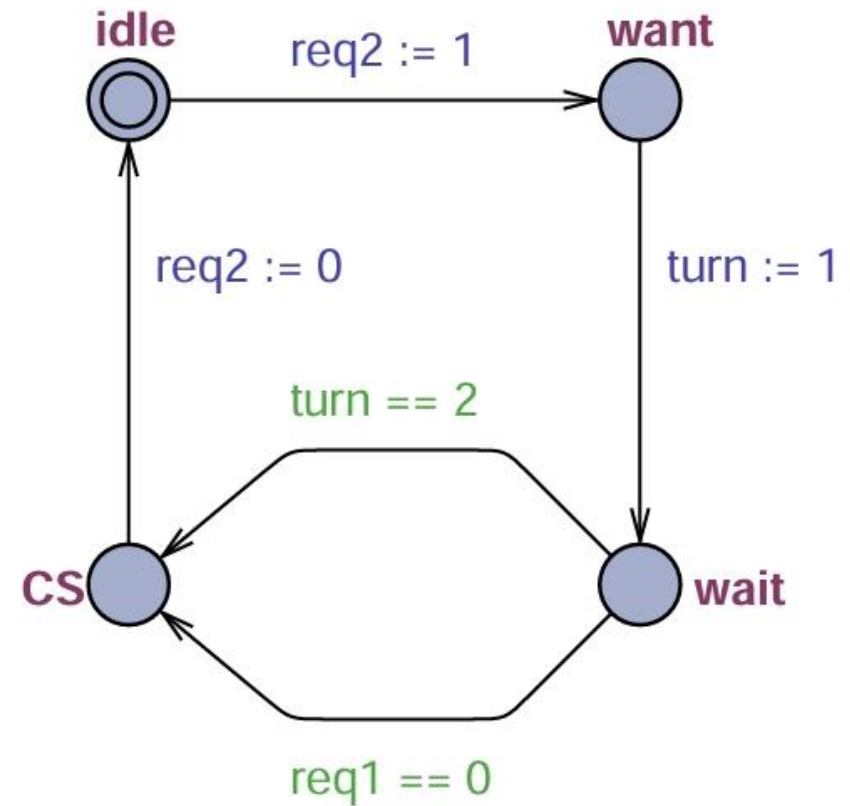
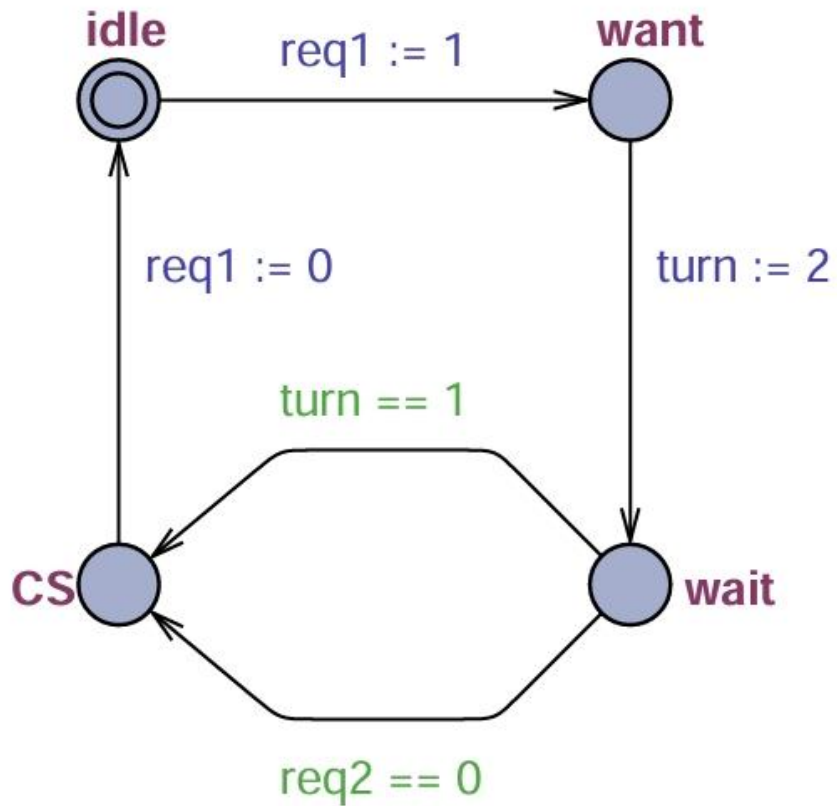
job2();

req2=0;

//and return to idle

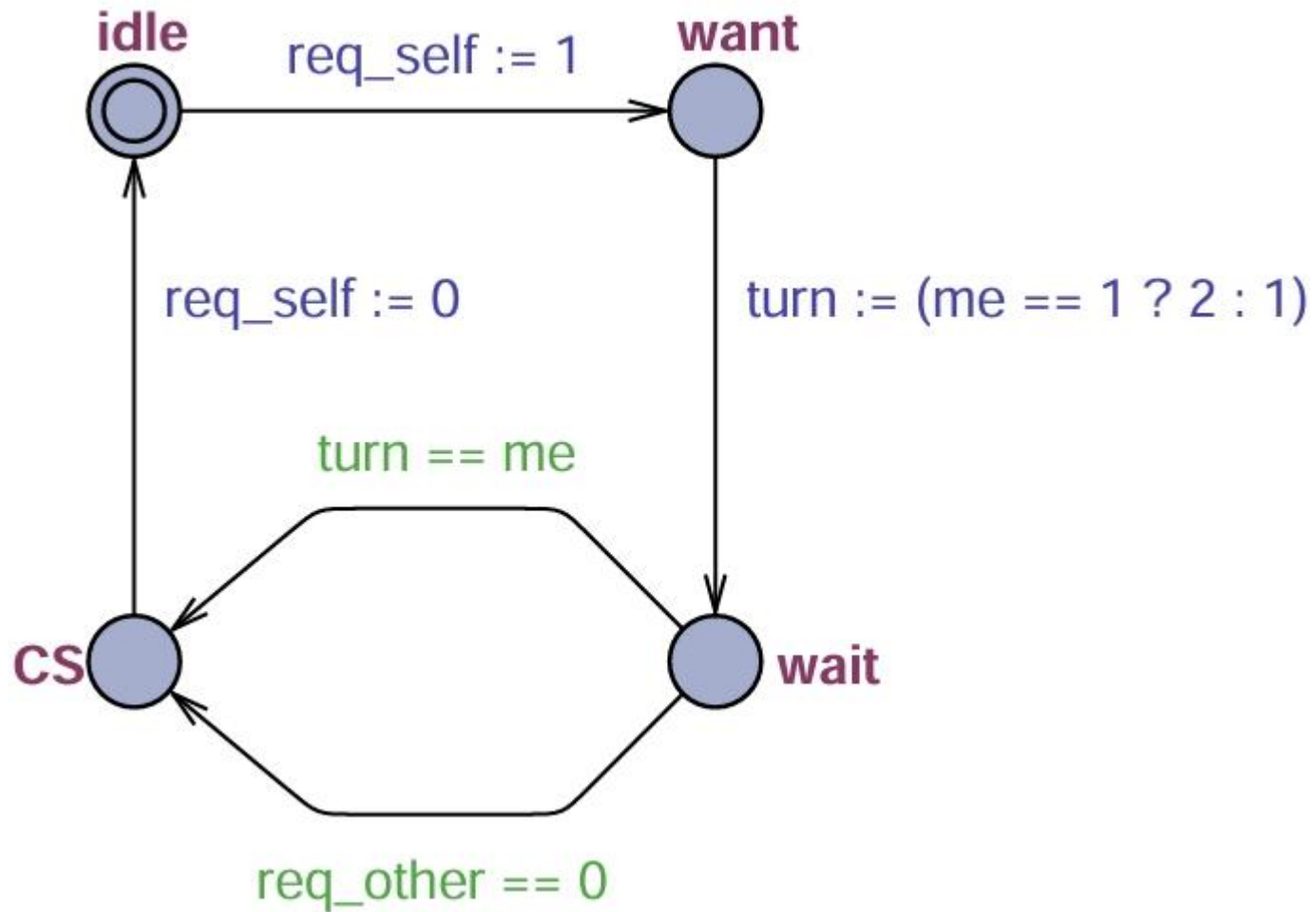


Example





Example





Lab parts

- Part 1: Warm-Up
 - ▶ Model 3 simple automata
 - ▶ Use verification for simple properties
- Part 2: Scheduling
 - ▶ Setting: Schedule jobs to CPUs
 - ▶ One automaton *per job* and *per CPU*
 - ▶ Determine minimal execution time
- Part 3: Deadlock detection
 - ▶ Model Buffer, Producer and Consumer from Ada lab
 - ▶ Use verifier to *find deadlocks*
 - ★ “Deadlock” means: Only time may pass (for all future)
 - ▶ Use simulator to analyze them
 - ▶ Remove all deadlocks



UPPSALA
UNIVERSITET

UPPAAL

END